

Control Structures

L18 - Selection Statements 2

Department of Computer Science
University of Pretoria

22 April 2024

Lecture 17 Recap:

- Ternary-IF Statements
- Nested Ternary-IF Statements
- Simple IF Statements

If-statements in C++ can be structured in various ways to achieve **multi-way branching**. One common approach is to use a series of if-else statements, which can be either *nested* or sequenced using *else if*. Both approaches can achieve the same outcome, but they differ in readability and structure. Consider categorizing the ages of individuals into three categories:

- young ($0 < 35$),
- middle-aged ($35 < 65$), and
- ancient ($65+$),

and printing the results to the screen.

- **Nested if-statements** occur when an if or else if block contains another if-statement. This nesting can continue to any level of depth as needed.
- The syntax for a nested if statement is as follows:

```
if( boolean_expression 1) {  
    // Executes when the boolean expression 1 is true  
    if( boolean_expression 2) {  
        // Executes when the boolean expression 2 is true  
    }  
}
```

Nested if statement code example:

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int age;
5     cout << "Type in an age [0 to 120]: ";
6     cin >> age;
7     if (age < 35) {
8         cout << "You are young" << endl;
9     } else {
10         if (age < 65) { // age >= 35 and age < 65
11             cout << "You are middle-aged" << endl;
12         } else {
13             cout << "You are ancient" << endl;
14         }
15     }
16     return 0;
17 }
```

Chained Else If-Statements

Alternatively, **chained if statements** can be used to simplify the nested if structure. The else if approach is more readable, especially when dealing with multiple levels of conditions.

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int age;
5     cout << "Type in an age [0 to 120]: ";
6     cin >> age;
7     if (age < 35) {
8         cout << "You are young" << endl;
9     } else if (age < 65) {
10         // age >= 35 and age < 65
11         cout << "You are middle-aged" << endl;
12     } else { // age >= 65
13         cout << "You are ancient" << endl;
14     }
15     return 0;
16 }
```

Write a boolean function called **isLeapYear()** that determines whether a year is a leap year or not. The function should have one input parameter, the year, and return true if the year is a leap year, and false if it is not. (**Note:** All years divisible by 4 are leap years. However, century years are an exception - they are only leap years when they are divisible by 400. For example, the year 1900 was not a leap year, but the year 2000 was one. Hint: Use the modulus operator % to test for divisibility.)

Exercise Solution:

```
1 #include <iostream>
2 using namespace std;
3 bool isLeapYear(int year) {
4     // Check if year is a leap year
5     if (year % 4 == 0) {
6         // Check if year is a century year
7         if (year % 100 == 0) {
8             // Check if divisible by 400
9             if (year % 400 == 0) {
10                return true;
11            } else {
12                return false;
13            }
14        } else {
15            return true;
16        }
17    } else {
18        return false;
19    }
20 }
```


Homework:

Now implement the main program where you ask the user to enter the year, then the main program calls the **isLeapYear()** function and the main program outputs whether the year is a leap year or not.

- There are times when if-statements have changed over time to a point where some code is duplicated or becomes redundant when writing if-statements.
- It is these cases the is becomes necessary to factorise this redundancy out of the statement and present cleaner and easier to read code.
- When changing code from one form to another, the notion of code equivalence is important to keep in mind.
- Factorising of if-statements can be done to *simplify* the code written. There are a number of factorisation techniques.
- These techniques fall into two broad categories, **factorisation of statements** and **factorisation of conditions**.

- **Factorisation of statements** removes the issue of code redundancy from the code-base.
- That is, repeated code that will be executed irrespective of the outcome of the evaluation of the condition will be consolidated and specified only once.
- This will increase the readability, understandability and maintainability of the code.
- Three types of factorisation can be applied to if-statements. These are **bottom-factorisation**, **top-factorisation** and **unnecessary factorisation**.

- **Bottom Factorisation** moves the statements that are duplicated at the end of the true and false blocks of an if-statement to after the if-statement.
- `<statement3>` is executed irrespective of what the result of the condition is. `<statement3>` can therefore be factored out and placed below the if-statement (refer to the right-hand column). Functionally the code on the left is equivalent to the code on the right in the table.

<pre>if (<condition>) { <statement1>; <statement3>; } else { <statement2>; <statement3>; }</pre>	<pre>if (<condition>) { <statement1>; } else { <statement2>; } <statement3>;</pre>
--	--

An example of bottom-factorisation is given below:

Code Example 1

```
#include <iostream>
int main() {
    int x;
    cout << "Enter a number: -";
    cin >> x;
    if (x > 100) {
        cout << x << endl;
        x += 2;
    } else {
        cout << x + 2 << endl;
        x += 2;
    }
    return 0;
}
```

Code Example 2

```
#include <iostream>
int main() {
    int x;
    cout << "Enter a number: -";
    cin >> x;
    if (x > 100) {
        cout << x << endl;
    } else {
        cout << x + 2 << endl;
    }
    x += 2;
    return 0;
}
```

- **Top-factorisation** is similar to bottom-factorisation in that the common code is the first statement(s) that is/are executed in both the true and false blocks.
- Unlike with bottom-factorisation, the statements can only be factored to before the beginning of the if-statement if the statement(s) **do not influence the condition** (important) of the if-statement.
- For the high-level code snippet given in the left column of the table below, `<statement1>` may only be removed resulting in the code given in column 2, if `<condition>` is not influenced by the statement.

<pre>if (<condition>) { <statement1>; <statement2>; } else { <statement1>; <statement3>; }</pre>	<pre><statement1>; if (<condition>) { <statement2>; } else { <statement3>; }</pre>
--	--

An example of top-factorisation is given below. As you can see, the factored statements do not effect the condition of the if statement:

Code Example 1	Code Example 2
<pre>#include <iostream> using namespace std; int main() { int x, result; cout <<"Enter a number: -"; cin >>x; if (x > 100) { result = x + 2; cout << result; } else { result = x + 2; cout << result + 2; } return 0; }</pre>	<pre>#include <iostream> using namespace std; int main() { int x, result; cout <<"Enter a number: -"; cin >>x; result = x + 2; if (x > 100) { cout << result; } else { cout << result + 2; } return 0; }</pre>

- **Unnecessary-factorisation** is concerned with the factorisation of the condition.
- Testing the condition takes time, unnecessary testing of the same condition slows down the execution of the program.
- In the left if statement, if the condition for the first if is the same as the condition for the else-if, and <statement1> has not changed any variable or value that will impact the outcome of the condition tested by the else-if, then it is not necessary to test the condition a second time. That is, <statement2> may be executed immediately after <statement1> without retesting the condition.

```
if (<condition>) {  
    <statement1>;  
} else if (<condition>) {  
    <statement2>;  
} else {  
    <statement3>;  
}
```

```
if (<condition>) {  
    <statement1>;  
    <statement2>;  
} else {  
    <statement3>;  
}
```


An example of unnecessary factorisation is given below:

Code Example 1	Code Example 2
<pre> #include <iostream> using namespace std; int main(){ int x; cout <<"Enter-a-number:-"; cin >>x; if (x < 10) { cout << x << endl; } else if (x < 10) { x += 10; cout << x << endl; } else{ x -=10; cout << x << endl; } return 0; } </pre>	<pre> #include <iostream> using namespace std; int main() { int x; cout <<"Enter-a-number:-"; cin >>x; if (x < 10) { cout << x << endl; } else{ x -=10; cout << x << endl; } return 0; } </pre>

- Condition swapping means that instead of testing for the condition to be true, you test for the complement (i.e., the opposite) of the condition.
- For example, if the condition were to be $(x > 10)$ the complement (!) of the condition would be:
 $(x < 10)$ or $!(x > 10)$.
- In the table below, when the complement of the the condition is applied to the if-statement on the left, the statements in the true block and the false block of the if-statement need to be switched. The right-hand side of the table is the result of applying this switch.

<pre>if (<condition>) { <statement1>; } else { <statement2>; }</pre>	<pre>if !(<condition>) { <statement2>; } else { <statement1>; }</pre>
--	---

An example of Conditional Swapping is given below:

Code Example 1	Code Example 2
<pre> #include <iostream> using namespace std; int main(){ int x, y; cout <<" Enter-2-Numbers:"; cin >>x >>y; if (x < y) { cout << x << " is - lesser"; } else { cout << y << " is - lesser"; } return 0; } </pre>	<pre> #include <iostream> using namespace std; int main() { int x, y; cout <<" Enter-2-Numbers:"; cin >>x >>y; if (x >= y) { cout << y << " is - lesser"; } else { cout << x << " is - lesser"; } return 0; } </pre>